Universidade Estadual de Mato Grosso do Sul
Ciência da Computação
Banco de Dados

Processamento e otimização de consultas

Prof. Nilton
nilton@comp.uems.br

Introdução

Lexicamente, sintaticamente e semanticamente

Representação interna da consulta

Plano de execução da consulta

Compilado ou interpretado

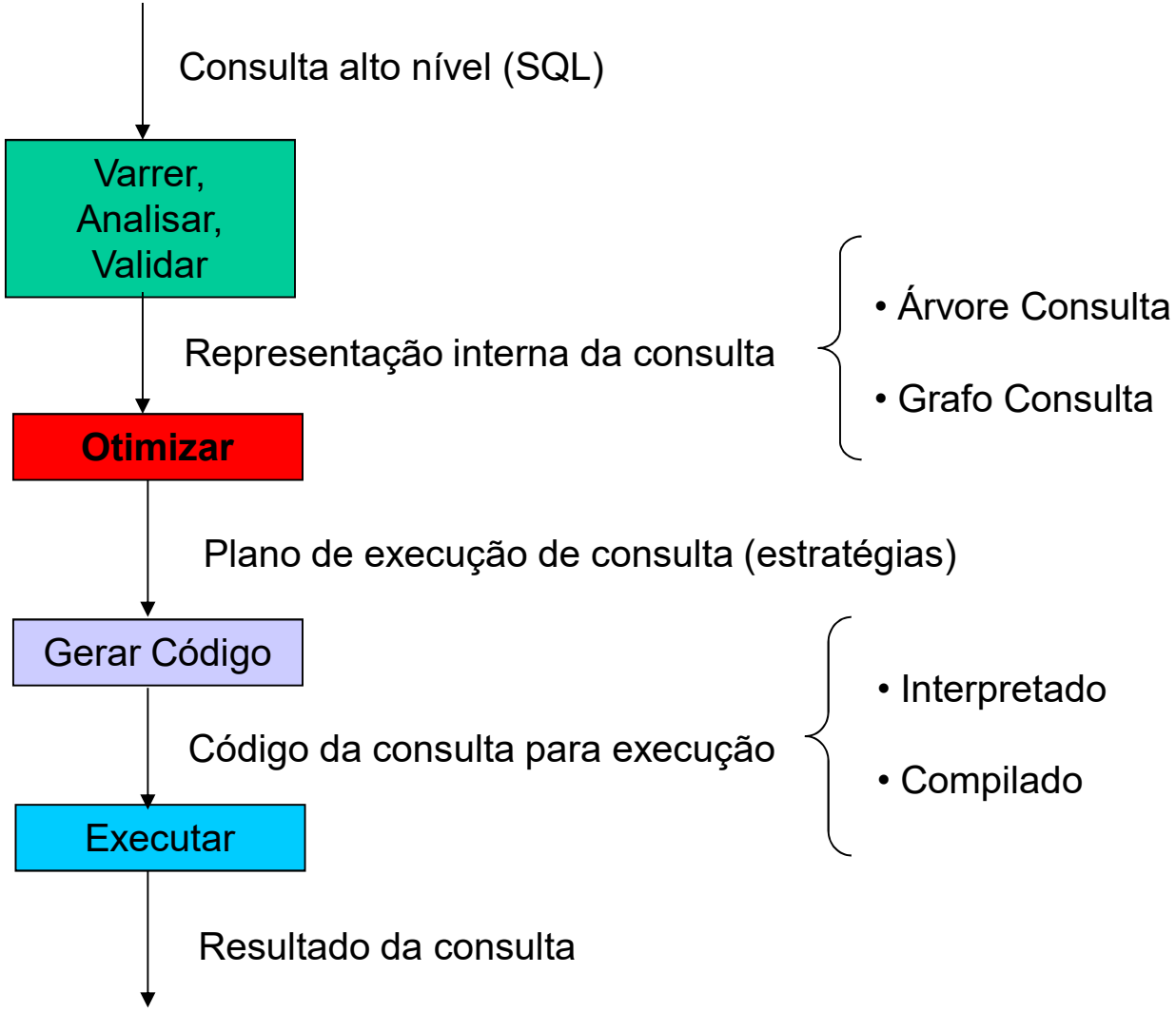
Uma consulta relacional é mais declarativa por natureza

Implementação chamando-se rotinas de acesso

Duas técnicas principais para otimização de consulta

- usando heurísticas
- por estimativa de custo

Passos do Processamento de Consultas



Otimizador de Consultas

Gera planos de execução de consulta (estratégias)

Tenta escolher o melhor plano (ótimo)

- Tempo?
- Disposição de informações de como os arquivos estão implementados e até mesmo sobre seu conteúdo?

Solução?

- A escolha é mais uma descrição mais precisa do que uma otimização

Por Heurísticas e por Custos

Técnicas principais:

- Regras Heurísticas

  Reordena as operações na árvore de consulta;

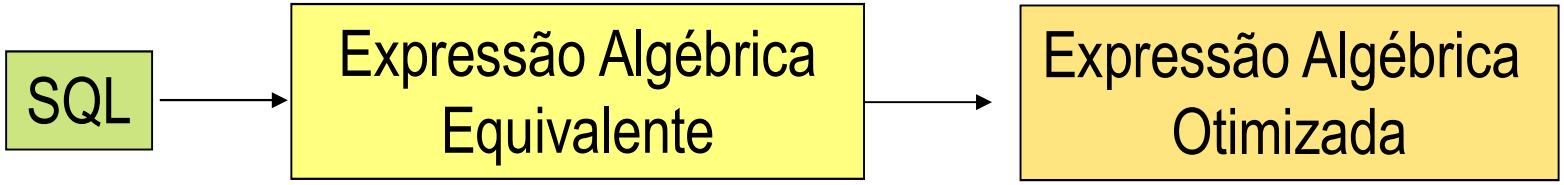
  Trabalha bem, mas não há garantias em todos casos.

- Estimativas de Custos

  Calcula-se o custo das várias estratégias de execução e escolhe-se a de menor custo;

Essas técnicas podem ser combinadas;

Traduzindo Consultas SQL para Álgebra Relacional



A expressão algébrica é representada em uma árvore de consulta;

Uma consulta é representada por um bloco de consulta;

- Um bloco contém:
  - A expressão SELECT-FROM-WHERE → traduzida em operadores algébricos;
  - Cláusulas GROUP BY, HAVING e funções agregadas → traduzida na álgebra relacional estendida.

Uma consulta pode ter vários blocos;

Exemplo de consulta

```
SELECT nomefim, nominic
FROM Emp
WHERE salario > ( SELECT max(salario)
                  FROM Emp
                  WHERE idDe=5 );
```

Bloco externo

Bloco interno

$\pi_{\text{nomefim, nominic}} (\sigma_{\text{salario} > c} (\text{Emp}))$

$\Im_{\text{max salario}} (\sigma_{\text{idDe}=5} (\text{Emp}))$

- A consulta tem dois blocos;
- O otimizador deve escolher uma estratégia de consulta para cada bloco;

Estratégias de Execução de Consultas

Implementam as operações do plano de execução de consulta;

Várias estratégias para uma única operação;

Uso de estruturas de armazenamento e meios de acesso;

Uso do principal algoritmo *Sort-Merge* para implementar as operações

Uso de armazenamento externo para comportar os arquivos de banco de dados

Ordenação Externa (*Sort-Merge*): dividir/conquistar

Maioria dos arquivos de banco de dados não cabem na memória principal;

Cláusulas e operações: ORDER BY, GROUP BY, DISTINCT, junção, união, intersecção, ... ;

Pode ser evitado se existir um índice primário ou de agrupamento

O espaço de *buffer* (cache SGBD) é dividido em *buffers* individuais;

Cada *buffer* tem o tamanho de um bloco de disco

Fase ordenação (*sort*)

Partes do arquivo são carregados no *buffer*, ordenados usando um algoritmo de ordenação interno e gravados em disco como sub-arquivos temporários.

A quantidade de sub-arquivos (n_{Run}) gerados é em função de:
números de blocos do arquivo (b) e tamanho do espaço do buffer disponível (n_{Buffer}), ou seja:
$n_{Run} = \lceil (b / n_{Buffer}) \rceil$

Fase intercalação (*merge*)

Os sub-arquivos são intercalados em um ou mais passos de intercalação.

O grau de intercalação (g_{Merge}) é o número de sub-arquivos que podem ser juntados em cada passo de intercalação. Esse grau é o menor entre ($n_{Buffer} - 1$) e n_{Run} .

O número de intercalações é definido por: $\lceil (\log_{gMerge} (n_{Run})) \rceil$

Ordenação Externa (*Sort-Merge*) - exemplo

Se o tamanho do arquivo é 3.000 blocos de disco, o número de *buffers* disponíveis é 5 blocos de disco, o grau de intercalação é 4, então pergunta-se

- Qual é o número de sub-arquivos gerados na fase de ordenação?

$$n_{Run} = \lceil (b / n_{Buffer}) \rceil = \lceil (3000/5) \rceil = 600$$

- Quantos passos são necessários na fase de intercalação para ordenar o arquivo?

$$n_{Pass} = \lceil (\log_{gMerge} (n_{Run})) \rceil = \lceil (\log_4 (600)) \rceil = 5$$

Se a cada passo serão intercalados 4 sub-arquivos temporários têm-se: no final do primeiro passo 150 (600/4) sub-arquivos temporários ordenados; do segundo 38 (150/4); do terceiro 10; do quarto 3 e finalmente no quinto 1.

Ordenação Externa (*Sort-Merge*)

O desempenho do algoritmo *Sort-Merge* pode ser medido no número de leituras e gravações de bloco de disco (entre disco e memória principal) antes que a ordenação inteira seja concluída

$$\underbrace{(2 * b)}_{\text{1ª divisão}} + \underbrace{(2 * b * (\log_{gMerge} n_{Run}))}_{\text{2ª conquista}}$$

No primeiro termo cada bloco de arquivo é acessado duas vezes, uma para leitura em um *buffer* da memória principal e uma para gravação no arquivo temporário ordenado no disco.

No segundo termo a cada intercalação uma quantidade de blocos de disco aproximadamente igual aos blocos do arquivo original *b* é lida e gravada.

Algoritmo *Sort-Merge*

```
atribua  i ← 1;
         j ← b;              {tamanho do arquivo em blocos}
         k ← n_B;     {tamanho do buffer em blocos}
         m ← ⌈(j/k)⌉ ;

{Fase de Ordenação}
enquanto (i ≤ m)
faça {
        lê próximos k blocos do arquivo no buffer ou, se houver menos de k blocos
        restantes, então lê nos blocos restantes;
        ordena os registros no buffer e grava como um subarquivo temporário;
        i ← i + 1;
}
{Fase de Intercalação: intercala subarquivos até que apenas 1 permaneça}
atribua   i ← 1;
          p ←  ⌈log_{k-1} m⌉     {p é o número de passos para a fase de intercalação}
          j ← m;
enquanto (i ≤ p)
faça {
    n ← 1;
    q ← ⌈(j/(k−1))⌉ ;          {número de subarquivos a gravar nesse passo}
    enquanto (n  ≤ q)
    faça {
            lê próximos k−1 subarquivos ou subarquivos restantes (do passo anterior)
                    um bloco por vez;
            intercala e grava como novo subarquivo um bloco por vez;
            n ←  n + 1;
    }
    j ← q;
    i ← i + 1;
}
```

Implementando a Operação Seleção (σ)

É implementada basicamente como um algoritmo de pesquisa

Existem muitas opções de métodos para execução;

- Algumas dependem de métodos de acessos usando arquivos ou certos tipos de condições de seleção.

Opções:

- Métodos de busca para Seleção Simples;
- Métodos de busca para Seleção Composta;

Operação de Seleção – Exemplos

OP1: $\sigma_{idE = '1234'}$ (Emp)

OP2: $\sigma_{idD > 5}$ (Dep)

OP3: $\sigma_{idDe = 5}$ (Emp)

OP4: $\sigma_{idDe = 5 \text{ AND } salario > 30000 \text{ AND } sexo = "F"}$ (Emp)

OP5: $\sigma_{idEtr = '1234' \text{ AND } idPtr = 10}$ (Trab)

Métodos de Busca para a Seleção Simples

Selecionar registros baseado numa condição de seleção;

Tipos de busca

- S1 : Linear (força bruta)
  - recupera cada registro do arquivo comparando com a condição de seleção;
- S2 : Binária
  - o arquivo está ordenado por um atributo chave que faz parte da condição de seleção numa comparação de igualdade (=);
- S3 : Uso índice primário (ou chave *hash*)
  - um atributo chave com índice primário (ou chave *hash*) que faz parte da condição de seleção numa comparação de igualdade (=) ;

Métodos de Busca para a Seleção Simples

Tipos de busca

- S4 : Uso índice primário para recuperar vários registros
  - nas condições de comparação (>, >=, <, <=) num atributo chave com índice primário;
- S5 : Uso índice cluster para recuperar vários registros
  - um atributo não chave com índice cluster que faz parte da condição de seleção numa comparação de igualdade (=);

Métodos de Busca para a Seleção Simples

Tipos de busca

- S6 : Uso índice secundário (B^+) em comparação de igualdade
  - recupera um único registro se o campo de índice for chave ou recupera vários registros se o campo de índice não for chave. Pode ser usado também nas operações >, >=, < ou <=.

O método S1 (pesquisa linear) se aplica a qualquer arquivo, mas os demais métodos dependem do caminho de acesso no atributo que faz parte da condição de seleção.

Métodos de Busca para a Seleção Composta(AND/OR)

Tipos de busca

- S7 : Seleção conjuntiva usando um índice individual
  - se um atributo que faz parte duma condição simples na condição conjuntiva tem um caminho de acesso que permite o uso de um dos métodos de S2 a S6, use essa condição para recuperar os registros e verifique se o registro recuperado satisfaz as demais condições simples;
- S8 : Seleção conjuntiva usando um índice composto
  - se atributos envolvidos nas condições de igualdade na condição de seleção conjuntiva também participam como índice composto (ou estrutura de *hash*);

Métodos de Busca para a Seleção Composta(AND/OR)

Tipos de busca

- S9 : Seleção conjuntiva por intersecção de ponteiros de registros
  - envolve índices secundários (ou outros caminhos de acesso) com atributos que fazem parte em condições simples e índices que inclui ponteiros de registros (ao invés de ponteiros de blocos), então cada índice pode ser usado para recuperar o conjunto de ponteiros de registros que satisfaz a condição individual.
  - A intersecção é dada pelo ponteiro de registro que satisfaz a condição conjuntiva.
  - Se apenas algumas condições tiverem índices secundários, cada registro recuperado deve ser testado com as demais condições restantes.

Considerações na implementação da operação de seleção (σ)

Não existe um caminho de acesso: uso de busca linear ou busca binária

Existe um caminho de acesso: utilize-o;

Na condição de seleção conjuntiva o otimizador deve escolher o caminho de acesso com menor custo de recuperação de registros;

A escolha está baseada na **seletividade** de cada condição;

Seletividade sobre condição de seleção conjuntiva(AND)

É a razão do número de registros que satisfazem a condição de seleção com o total de registros do arquivo;

É um número entre 0 e 1. **Zero** nenhum registro satisfaz. **UM** todos registros satisfazem;

A seletividade para condição de igualdade em um atributo **chave** é:

sl = 1 / |r(R)|, onde |r(R)| é o número registros do arquivo

A seletividade para condição de igualdade em um atributo **não chave** com ***i*** valores distintos é:

sl = (|r(R)| / i) / |r(R)| ou 1 / i, considerando registros iguais ou uniformemente distribuídos

Em geral o número de registros que satisfazem a condição de seleção é:

cs = |r(R)| * sl, onde cs é cardinalidade de seleção

A condição de seleção com o menor valor estimado será utilizada primeiro;

Uso de estimativa quando não é possível calcular. Estão guardadas no catálogo do SGBD.

Seletividade

condição de seleção conjuntiva(AND) x condição de seleção disjuntiva(OR)

Uma condição de seleção disjuntiva é muito mais difícil de processar e otimizar se compararmos com uma condição de seleção conjuntiva;

Por exemplo,

OP4: $\sigma_{\text{idDe = 5 OR salario > 30000 OR sexo = "F"}}$ (Emp)

+ o resultado é a união dos registros selecionados em cada condição individual
+ assim a ordem de processamento de cada seleção simples é independentemente.

Implementando a Operação de Junção

Tempo gasto elevado de processamento;

Vamos ver a Junção Natural ou EQUIJUNÇÃO

- Junção Dupla (2 arquivos);
- Junção Múltipla (vários arquivos); Não será estudada

Número de caminhos possíveis para implementar a junção múltipla é muito grande;

Implementando outras Junções

- Junção à direita, à esquerda, completa;

Operação de Junção - Exemplos

$R \ |X|_{A=B} \ S$

OP6 : $Emp \ |X|_{idDE=idD} \ Dep$

OP7 : $Dep \ |X|_{idEchefe=idE} \ Emp$

Junção - Métodos de Implementação

J1 : Junção de laço aninhado (força bruta)

para cada registro do 1º arquivo teste com todos os registros do 2º arquivo

J2 : Junção de único laço (usando estrutura de acesso para recuperar registros que combinam): supor que o 1º arquivo tem um índice (ou chave *hash*) em um dos atributos de junção;

para cada registro do 2º arquivo compara-o com o 1º arquivo usando o índice;

os registros que combinam com o atributo de junção formarão os resultados

J3 : Junção ordenação-intercalação *(sort-merge)*

Os arquivos estão ordenados fisicamente por atributos de junção;

Os arquivos serão varridos simultaneamente;

O algoritmo emparelha os valores e compara-os com os atributos de junção;

Se os arquivos não estiverem ordenados, ordena-os

Junção - Métodos de Implementação

J4 : Junção de partição-hash

Os registros dos arquivos são particionados em arquivos menores usando uma mesma função *hash* nos atributos de junção

Fase particionamento: o arquivo menor é particionado (supondo que ele caiba na memória principal) e seus registros são colocados em *buckets hash*

Fase sondagem: o outro arquivo é particionado e cada registro é invesigado no *bucket* apropriado e combinado com registros correspondentes

Todas as técnicas usam acesso em blocos inteiros de disco, em vez de acesso a registros individuais. Assim o tamanho de cada bloco pode ser ajustado de acordo com a memória disponível;

Junção – Comportamento

(Espaço disponível *buffer* e o fator de seleção da junção no desempenho da junção)

OP6: Emp $|X|_{idDE=idD}$ Dep

50 departamentos em 10 blocos de disco (b_D=10)

6000 empregados em 2.000 blocos de disco(b_E=2.000)

buffer de tamanho 7 blocos na memória

Supondo que um *buffer* tem o mesmo tamanho de um bloco de disco

Desenvolvimento da J1 (*força bruta*)

Buffer : Um bloco para o resultado, outro para o loop interno e o resto para o loop externo

Número de blocos lidos pelo arquivo externo : b_{ext}

Número de vezes (n_B - 2) que os blocos do arquivo externo são carregados para memória principal:

$$teto(b_{ext} / (n_B - 2))$$

Numero de blocos lidos pelo arquivo interno :

$$b_{int} * teto(b_{ext} / (n_B - 2))$$

Cálculo do nº total de acessos/leituras a blocos

$$b_{ext} + (\lceil b_{ext} / (n_B - 2) \rceil * b_{int})$$

<u>Custo 1</u>:

Usando o arquivo de Empregados no loop externo:

Num. acessos = 2.000 + 10 * teto(2.000 / (7 - 2)) = 6.000

<u>Custo 2</u>:

Usando o arquivo de Departamentos no loop externo:

Num. acessos = 10 + 2.000 * teto(10 / (7 - 2)) = 4.010

Desenvolvimento da J2 (*índice ou chave hash*)

OP7: Dep $|X|_{idEchefe=idE}$ Emp

6.000-50 = 5.950 empregados serão descartados pela junção

Existe um índice secundário de 4 níveis sobre idE (empregado)
Existe um índice secundário de 2 níveis sobre idEchefe (departamento)

Usando a J2, o núm. de acessos a blocos é aprox.

$$b_{ext} + (r_{ext} * (x_{int} + 1))$$

Custo 1:
Arquivo de Empregados + Departamento por índice:

2.000 + (6.000 * (2 + 1)) = 20.000 acessos

Custo 2:
Arquivo de Departamentos + Empregados por índice:

10 + (50 * (4 + 1)) = 260 acessos

Desenvolvimento da J2 (*índice ou chave hash*)

A segunda opção é mais eficiente porque o

fator de seleção de junção (**fração de registros em um arquivo que será juntada com os registros do outro arquivo**)

de Departamento com relação à condição de junção idEchefe=idE é **1** porque todos (100%) registros de Departamento tem um chefe e então serão juntados com algum de Empregado;

enquanto que o fator de seleção de junção de Empregado é **0,00833** (50/6.000), ou seja, apenas 0,8% dos registros em Empregados serão juntados

O menor arquivo ou o arquivo que tenha um par para todos os registros (o arquivo com fator de seleção de junção alto) deve ser utilizado no laço de junção (externo).

Desenvolvimento da J3 (*junção ...-merge*)

É muito eficiente se os dois arquivos já estiverem ordenados pelos atributos de junção,

Com um custo de $(b_{R1} + b_{R2})$, ou seja, a soma do número de blocos acessados dos dois arquivos

Ex: OP6 ou OP7 é 10 + 2000 = 2010 acessos

A estimativa do custo de ordenação de um arquivo é $b \log_2 b$, assim

Sem ordenação nos dois arquivos, o custo de acesso a blocos desta junção pode ser estimado por

$$(b_{ext} * (\log_2 b_{ext}) + b_{int} * (\log_2 b_{int})) + (b_{R1} + b_{R2})$$

Desenvolvimento da J4 (*junção partição-hash*)

É muito eficiente para arquivos ordenados ou não

Uma única etapa para cada arquivo é suficiente
(melhor caso)

O custo estimado da junção partição-hash em acessos a blocos é

$$3 * (b_R + b_S) + b_{RES}$$

Na fase de partição cada registro é lido uma vez e escrito de volta no disco uma vez.

Na fase de intercalação cada registro é lido uma segunda vez e feito a escrita da união.

Desenvolvimento da J4 (*junção partição-hash*)

Difícil garantir uma função *hash* de particionamento uniforme, ou seja, que os tamanhos das partições sejam quase iguais

Se uma partição for muito grande não caberá no espaço de memória disponível (n_B) para a segunda fase do algoritmo

Se $\mathbf{n_B > (b_R + 2)}$ for satisfeita, então o arquivo R pode ser particionado inteiramente na memória, ficando o custo de acessos a blocos em

$$(b_R + b_S) + b_{RES}$$

Desempenho da Junção

Qual espaço disponível para o buffer?

- O espaço para buffer tem efeito importante em vários algoritmos de junção;

Qual o fator de seleção de junção?

- É a razão de registros do arquivo que serão juntados com o outro arquivo ($\neq$seletividade de junção);

Implementação de outras operações de junção

Junção externa à esquerda

Junção externa à direita

Junção externa completa

pesquisar...

Implementando a Projeção (π) e Operações de Conjunto

Operação Projeção

- A chave faz parte da lista de atributos na projeção em R
  - Arquivo resultante tem mesmo tamanho do arquivo R
- A chave não faz parte da lista de atributos na projeção
  - Eliminação de duplicatas por ordenação ou tabelas hash

Operações de Conjuntos

- Produto cartesiano é a mais cara de implementar
- União, intersecção e diferença, também são caras
  - São aplicadas com compatibilidade de união
  - É usual utilizar a técnica de *sort-merge*
  - Uso também de *hashing*

Implementando Funções Agregadas e Agrupamento

Geralmente é preciso percorrer o arquivo inteiro, mas não é preciso quando:

- Existe índice descendente no atributo da função MAX;
- Existe índice ascendente no atributo da função MIN;

Índices podem ser utilizados com COUNT, AVG e SUM, mas somente com índices densos

Para cláusula GROUP BY deve-se particionar o arquivo em grupos e aplicar em cada grupo o operador de agregação separado;

- Ordenação ou *Hash* nos atributos de agrupamento;
- Índices clusterizados (os registros já estarão agrupados) ;

Combinação de Operações usando Pipelining

A consulta traduzida em uma expressão da álgebra relacional descreve uma seqüência de operações

Uso de arquivo temporário quando executa-se uma operação de cada vez ocasiona custo adicional

Solução é combinar as operações gerando código de execução dinâmicos

Pipelining: uma saída de uma operação é entrada para uma outra

- Exemplo
  - Se uma operação de junção executa após duas operações de seleção,
  - então os registros produzidos em cada seleção são fornecidos como entrada para a junção

Otimização de Consultas

usando

Heurísticas

Usando Heurísticas em Otimização de Consultas

Técnicas de otimização usam regras heurísticas para melhorar o desempenho das consultas, onde a representação interna da consulta inicial é modificada;

Representação interna

- **Árvore de Consulta**: representa uma expressão da álgebra relacional ou estendida;
- **Grafo de Consulta**: representa uma expressão do cálculo relacional;

Notação Árvore de Consulta

Representa a expressão da álgebra relacional e suas extensões;

NÓS FOLHAS estão as relações de entrada;

NÓS INTERNOS estão os operadores;

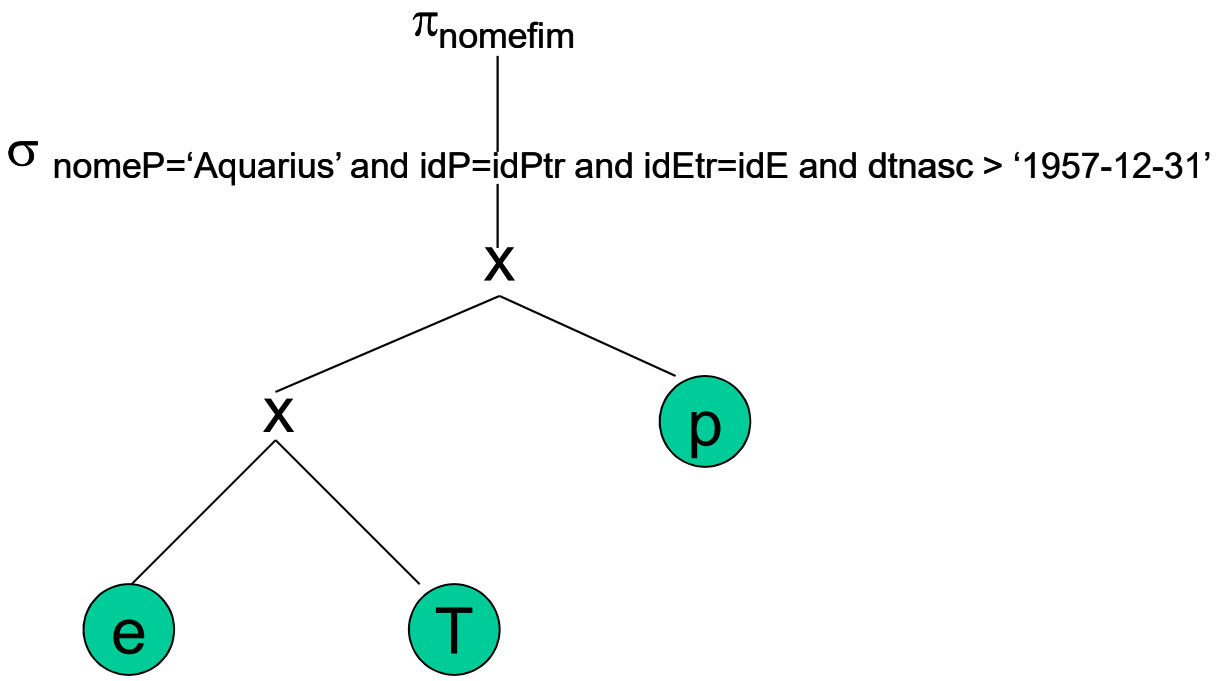
A ordem de execução da consulta está especificado;

- A execução inicia pelos operadores próximos às folhas, subindo até chegar ao nó raiz, produzindo o resultado da consulta;

Exemplo Árvore de Consulta

```
SELECT nomefim
FROM Emp E, Trab T, Proj P
WHERE nomeP = 'Aquarius' AND idP=idPtr AND
idEtr=idE AND dtnasc > '31/12/57'
```

$\pi_{nomefim}$ (σ nomeP='Aquarius' and idP=idPtr and idEtr=idE and dtnasc > '1957-12-31' ((Emp X Trab) X Proj))



Por que Árvore de Consulta

Na prática é preferível o uso da representação em árvore, ao invés do grafo porque o otimizador precisa da ordem de execução das operações na consulta, o que o grafo não representa;

Exemplo de uma Regra Heurística

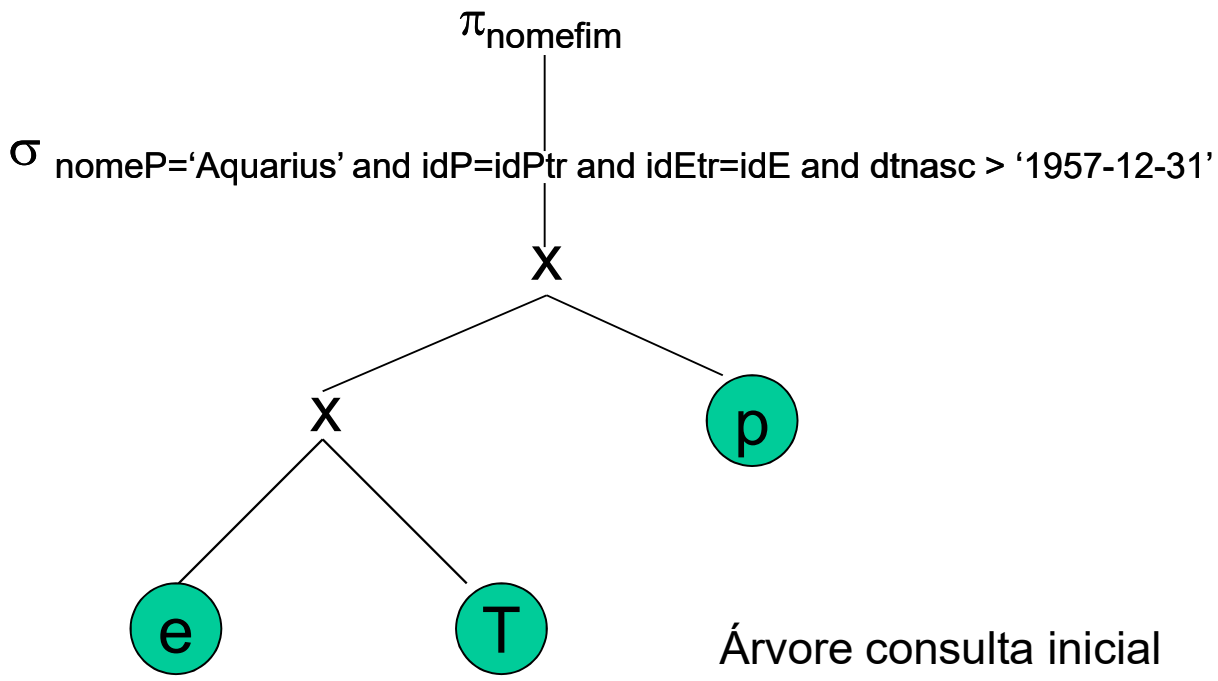
Aplicar as operações de seleção e projeção antes da operação de junção, porque é reduzido o tamanho dos arquivos, o que torna menos custoso para a junção, que é multiplicativa;

Otimização Heurística de Árvores de Consulta

```
SELECT nomefim
FROM Emp E, Trab T, Proj P
WHERE nomeP = 'Aquarius' AND idP=idPtr AND
idEtr=idE AND dtnasc > '31/12/57'
```

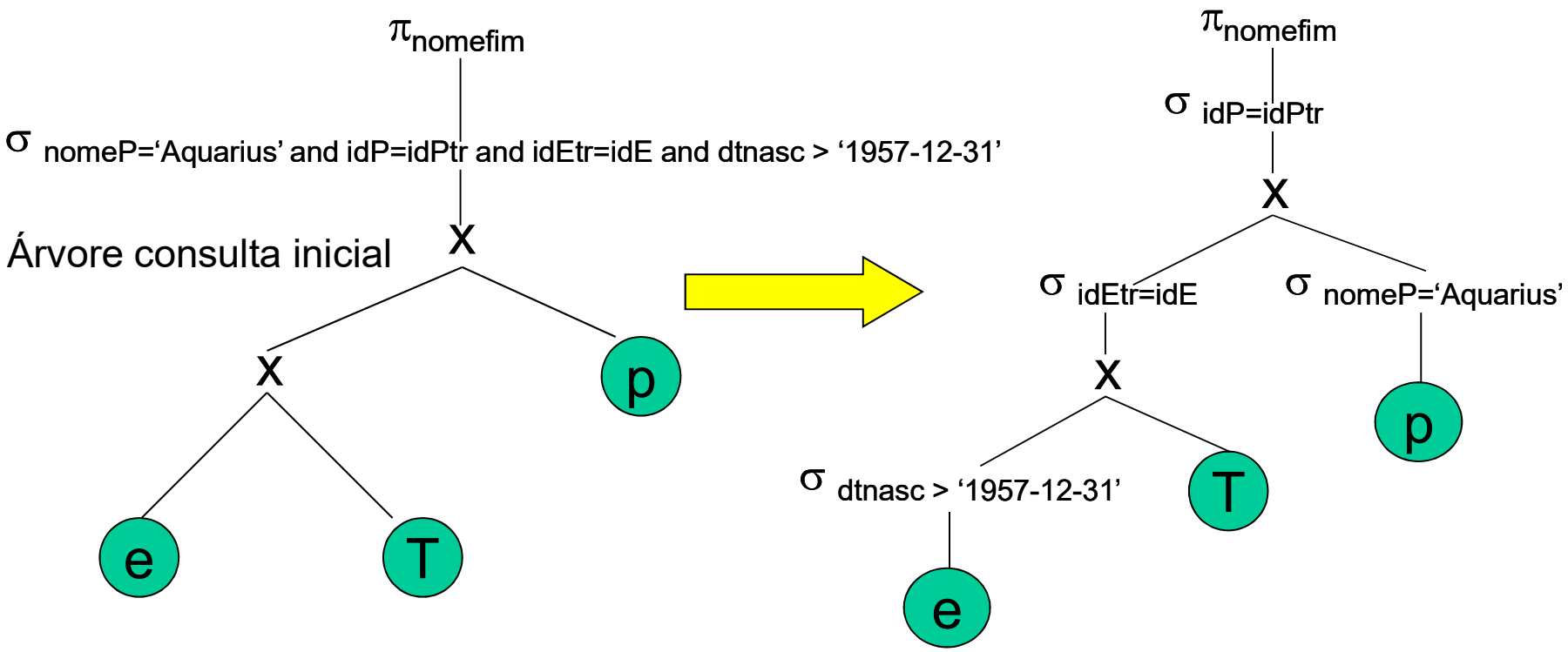
$$\pi_{nomefim}\ (\sigma_{nomeP='Aquarius'\ and\ dtnasc>'31/12/57'}((Proj) \bowtie_{idP=idPtr} (Trab) \bowtie_{idEtr=idE} (Emp)))$$

Otimização Heurística - Árvore Canônica

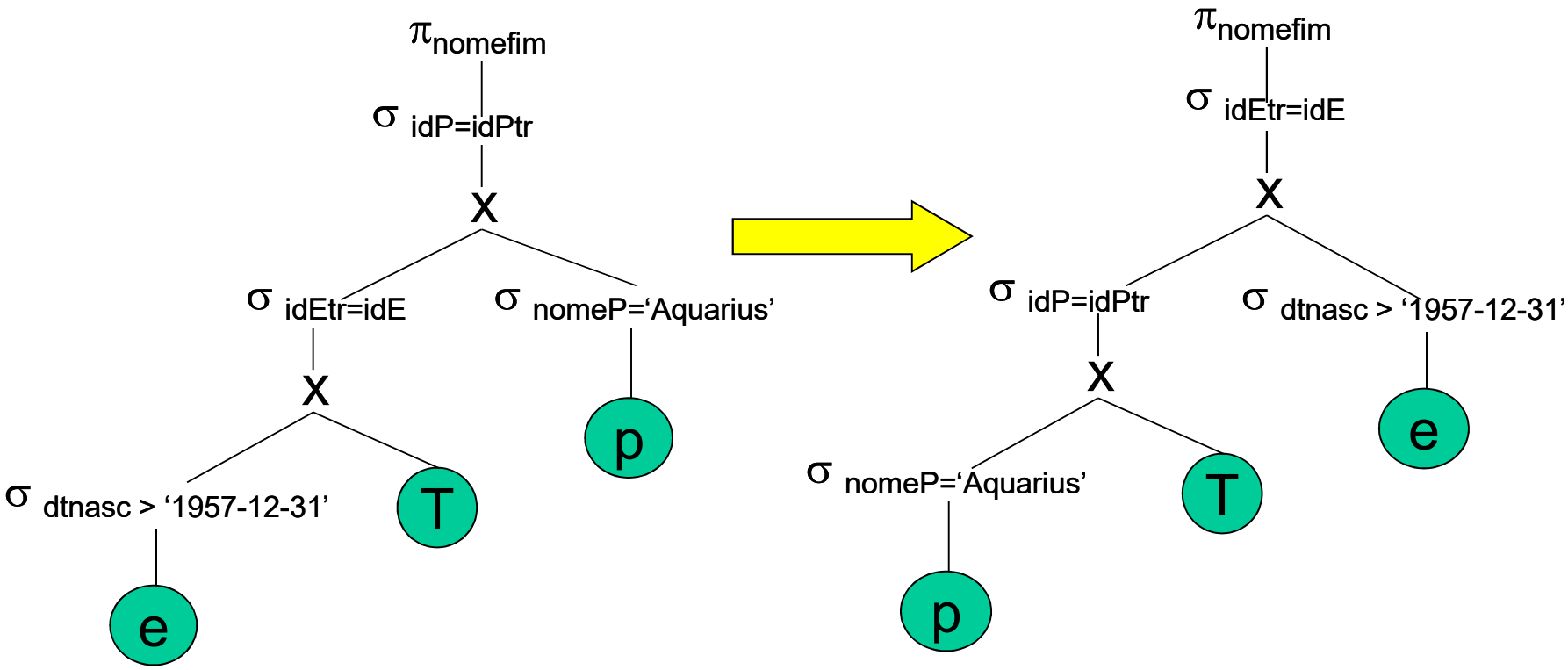


Árvore consulta inicial

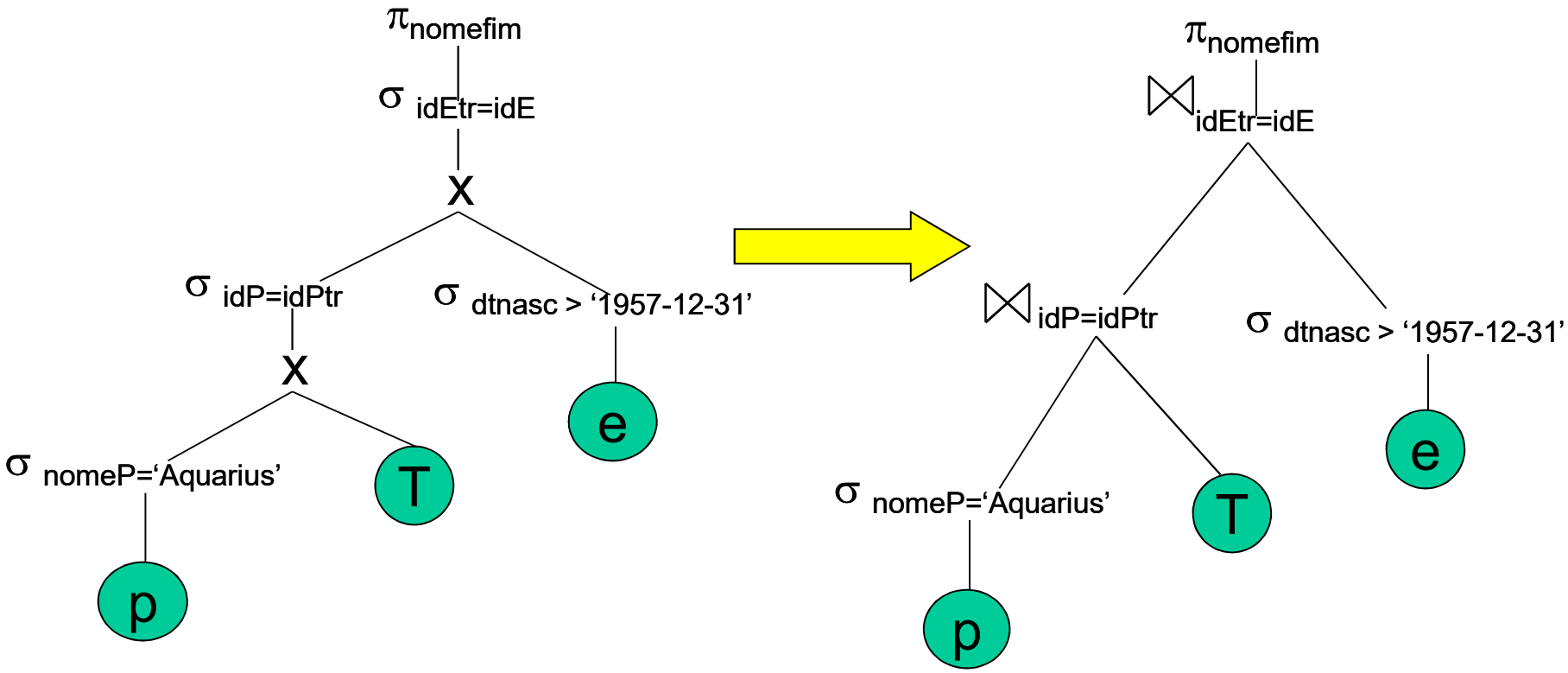
Descendo as Condições de Seleção



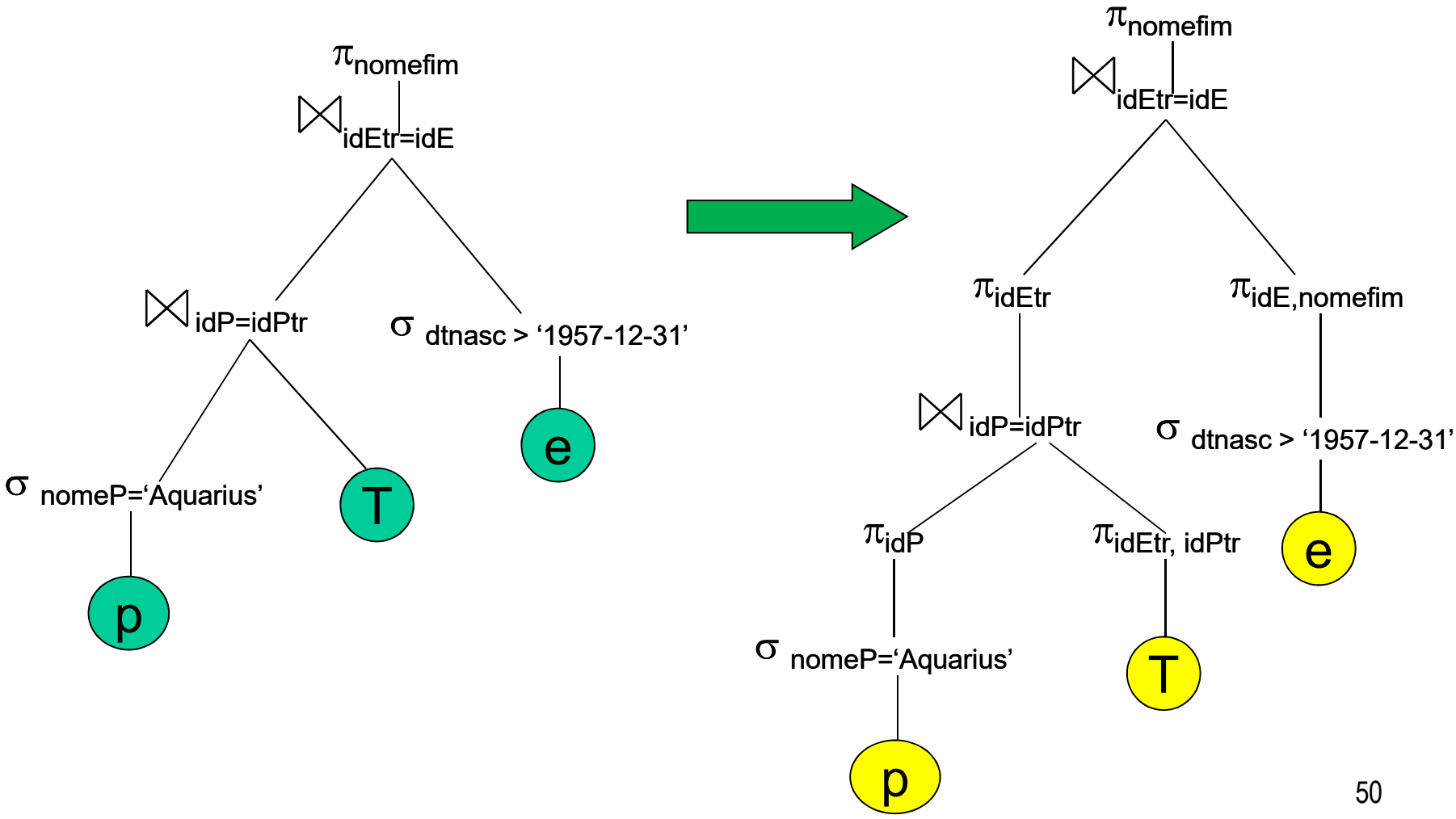
Mudando a posição das tabelas



Substituindo produto cartesiano + seleção



Adicionando projeções^{Fim}



Algumas regras de transformação para operações da álgebra relacional

1. seleção em cascata se seleção conjuntiva
2. seleção comutativa
3. projeção em cascata
4. comutar seleção com projeção
5. junção e produto cartesiano são comutativos
6. comutar seleção com junção ou produto cartesiano
7. comutar projeção com junção ou produto cartesiano
8. união e intersecção comutativas
9. associatividade de ><, X, intersecção e união
10. comutar seleção com união, intersecção ou diferença
11. comutar projeção com união
12. trocar seleção e X por ><
13. aplicar leis DeMorgan para condições

1-Cascata de σ

A condição de seleção conjuntiva pode ser quebrada em sequências individuais de operações σ

$$\sigma_{c1 \text{ and } c2 \text{ and } \ldots \text{ and } cn}(R) \equiv \sigma_{c1}(\sigma_{c2}(\ldots(\sigma_{cn}(R))\ldots))$$

2-Comutatividade de σ

A operação σ é comutativa

$$\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$$

3-Cascata de π

Todas as operações exceto a última podem ser ignoradas

$$\pi_{list1}(\pi_{list2}(\ldots(\pi_{listn}(R))\ldots)) \equiv \pi_{list1}(R)$$

4-Comutando σ com π

Podem ser comutadas quando a condição de seleção *c* envolve somente aqueles atributos A1, ..., An na lista de projeção

$$\pi_{A1, A2, ..., An}(\sigma_c(R)) \equiv \sigma_c(\pi_{A1, A2, ..., An}(R))$$

5-Comutando junção (ou produto cartesiano)

$$R \bowtie_c S \equiv S \bowtie_c R$$

$$R \times S \equiv S \times R$$

- A ordem dos atributos na tabela resultante pode não ser a mesma, mas pela definição de relação isso não é importante

6-Comutar seleção com junção (ou produto cartesiano)

Se todos os atributos da condição de seleção *c* envolve só os atributos de uma das relações que estão juntadas

$$\sigma_c (R \bowtie S) \equiv (\sigma_c (R)) \bowtie S$$

Da mesmo forma se *c* puder ser escrita como
(c1 AND c2) e *c1* composto apenas de atributos de uma das relações e *c2* da outra então

$$\sigma_{c1 \text{ and } c2} (R \bowtie S) \equiv (\sigma_{c1} (R)) \bowtie (\sigma_{c2} (S))$$

7-Comutar projeção com junção (ou produto cartesiano)

A lista de projeção sendo $L=\{A_1, ..., A_n, B_1, ...,B_m\}$, onde $A_1, ..., A_n$ são atributos de R e $B_1, ...,B_m$ são atributos de S, se a condição de junção *c* envolve só atributos em *L*, as duas operações podem ser comutadas

$$\pi_L (R \bowtie_c S) \equiv (\pi_{A1,...,An} (R)) \bowtie_c (\pi_{B1,...,Bm} (S))$$

•Caso na condição de junção existam alguns atributos que não conste em L, eles precisam ser incluídos nessa lista antes de executar a comutação e no final uma projeção é necessária. Para X não existe esta preocupação porque não tem condição *c*.

8-Comutatividade de união (ou intersecção)

As operações $\cup$ e $\cap$ são comutativas, mas a diferença não

$$R \cup S \equiv S \cup R$$

$$R \cap S \equiv S \cap R$$

$$R - S \neq S - R$$

9-Associatividade de ><, X, intersecção e união

Representando qualquer uma dessas operações por θ, teremos:

$$(R \ \theta \ S) \ \theta \ \ T \equiv R \ \theta \ (S \ \theta \ T)$$

10-Comutar seleção com união, intersecção ou diferença

Representando qualquer uma dessas operações por θ, teremos:

$$\sigma_c (R \ \theta \ S) \equiv (\sigma_c (R)) \ \theta \ (\sigma_c (S))$$

11-Comutar projeção com união

$$\pi_L(R \cup S) \equiv (\pi_L(R)) \cup (\pi_L(S))$$

12-Trocar “seleção e X” por “><“

Se a condição *c* de σ seguida por X corresponde a condição junção, a conversão é possível

$$(\sigma_c (R \times S)) \equiv (R \bowtie_c S)$$

13-Aplicar leis DeMorgan para condições

$$\text{not } (c1 \text{ and } c2) \equiv (\text{not } c1) \text{ or } (\text{not } c2)$$

$$\text{not } (c1 \text{ or } c2) \equiv (\text{not } c1) \text{ and } (\text{not } c2)$$

Algoritmo de Otimização Heurística Algébrica

a. Por 1 quebrar seleções com condições conjuntivas

Dará liberdade para movimentar seleções para baixo

b. Por 2, 4, 6, 10 ver comutatividade da seleção com outras operações

Movimentar cada seleção para o mais baixo possível

- se condição envolver um atributo de uma tabela é uma condição de seleção, então a condição deve ficar acima do nó folha dessa tabela
- se condição envolver um atributo de duas tabelas é uma condição de junção, então a condição deve ficar acima do nó folha dessas tabelas

c. Por 5 e 9 usar comutatividade e associatividade de operações binárias

Reorganizar os nós folha da árvore

1- deixar as tabelas com as seleções que produzem a menor quantidade de registros para serem executas primeiro

2-garantir que a ordenação dos nós folhas não cause operações de produto cartesiano

Algoritmo de Otimização Heurística Algébrica

d. Por 12 combinar produto cartesiano com seleção como junção

e. Por 3, 4, 7, 11 ver cascata e comutação de projeção com outras operações

Desmembrar e mover a lista de atributos de projeção para o mais baixo possível, criando novas operações de projeção

- Manter apenas os atributos necessários no resultado da consulta e nas operações subseqüentes

f. Identifique subárvores que representam grupos de operações que podem ser executados por uma rotina de acesso[fim].

Notação Grafo de Consulta

Representa a expressão do cálculo relacional;

NODOS representam as relações de entrada, conhecidos como nodos relação (círculo simples);

NODOS representam valores constantes, conhecidos como nodos constantes (círculo duplo);

ARESTAS representam as condições de seleção e junção;

Os atributos estão representados em retângulos acima da relação;

A ordem de execução da consulta não está especificado;

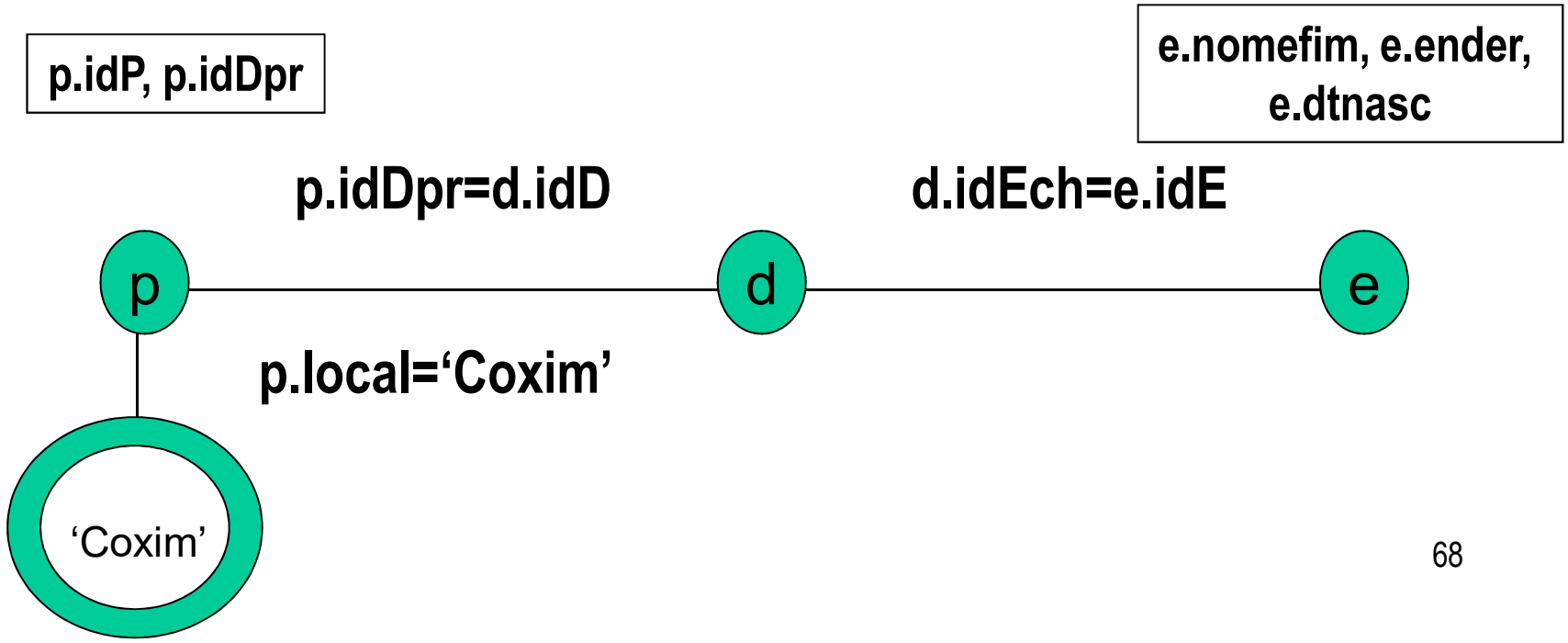
Cada consulta é representada por um grafo simples;

Existem técnicas de otimização em grafos.

Notação Grafo de Consulta

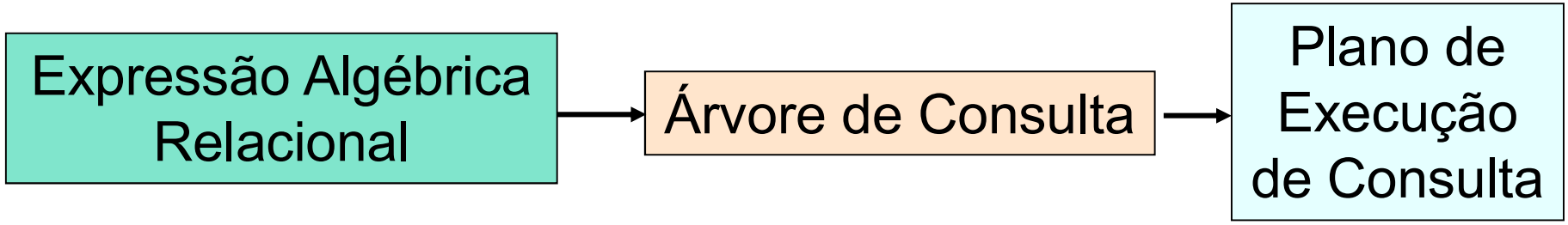
SELECT p.idP, p.idDpr, e.nomefim, e.ender, e.dtnasc
FROM Proj as p, Dep as d, Emp as e
WHERE p.idDpr = d.idD and d.idEch = e.idE and p.local = ‘Coxim’

$$\pi_{idP,\ idDpr,\ nomefim,\ ender,\ dtnasc}\ (\sigma_{local='Coxim'}((Proj) \bowtie_{idDpr=idD} (Dep) \bowtie_{idEch=idE} (Emp)))$$

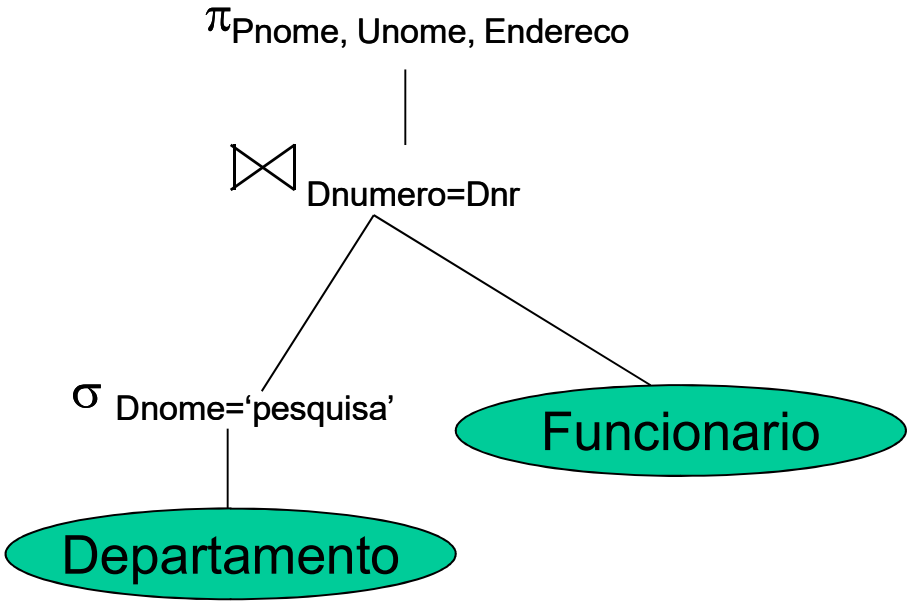


Convertendo Árvores de Consulta em Planos de Execução de Consulta

O plano de execução de consulta possui informações dos métodos de acesso disponíveis para cada tabela, bem como dos algoritmos para a execução dos operadores relacionais representados na árvore de consulta.



Exemplo de um Plano de Execução de Consulta



Para o plano para a árvore o otimizador poderia escolher:

- Busca de índice para o operador SELEÇÃO em Departamento (supondo que exista um)
- Algoritmo de JUNÇÃO DE LOOP ÚNICO sobre os resultados do operador SELEÇÃO em Departamento (supondo a existência do índice no atributo Dnr de Funcionário)
- Varredura do resultado da JUNÇÃO para o operador PROJEÇÃO

No plano ainda poderia mencionar o uso da técnica materializada ou canalizada

Otimização de Consultas

usando

Estimativas de Custos

Estimativas de Custos

Um otimizador de consultas não depende unicamente de regras heurísticas, numa otimização mais elaborada também,

1- Estima e compara custos da execução de uma consulta usando **diferentes estratégias de execução** e **algoritmos**

2- Escolhe a estratégias com custo estimado mais baixo

As estimativas de custos devem ser a mais precisa possível

O otimizador deve limitar o número de estratégias de execução, caso contrário muito tempo será gasto para construir as estimativas para as muitas estratégias de execução

Para não atrasar a execução de uma consulta

uma otimização mais elaborada para consultas compiladas

uma otimização parcial para consultas interpretadas

Estimativas de Custos

A abordagem é conhecida como Otimização de Consulta Baseada em Custo

- Usa-se técnicas tradicionais de otimização, onde pesquisa-se num espaço de soluções, a solução do problema que minimiza a **função (custo)** objetivo;
- As **funções de custos** usadas na otimização são estimativas e não funções exatas, assim o otimizador poderá escolher uma estratégia de execução da consulta que não é uma ótima;

Componentes de Custo para Execução Consulta

1. Custo de acesso a disco
   - Custo para transferir (ler e gravar) blocos de dados entre o disco e os *buffers* da memória principal
   - Depende dos métodos de acesso de um arquivo
   - Da forma que os blocos de arquivo estão armazenados (mesmo cilindro ou espalhados)
2. Custo de armazenamento temporário
   - Arquivos intermediários gerados por uma estratégia de execução
3. Custo de computação
   - Custo da execução das operações da consulta
4. Custo de uso da memória
   - Custo sobre o número de buffers da memória principal necessário durante a execução da consulta
5. Custo de comunicação
   - Custo de envio da consulta e resultados do local do banco de dados até o local do terminal

Componentes de Custo para Execução Consulta

Para BD grandes a ênfase é minimizar o **acesso a disco**;

Para BD pequenos, onde os dados podem ser completamente armazenados em memória principal, a ênfase é minimizar o **custo de computação**;

Para BD distribuídos, onde muitos locais estão envolvidos, a ênfase é o **custo de comunicação**;

Algumas funções de custo consideram o **custo de acesso a disco**, pois é difícil incluir todos componentes de custos numa função de custo, pela dificuldade de atribuir pesos adequados aos componentes de custos.

Informação de Catálogo para as Funções de Custo

Número de registros, tamanho do registro médio, número de blocos, fator de bloco, índices primários/secundários e atributos de indexação, número de níveis de um índice multiníveis, número de blocos de índice de primeiro nível, número de valores distintos de um atributo e sua seletividade

Informações sobre nível de índice é fácil manter porque não muda com freqüência, porém outras podem mudar

nº registros (inserção/exclusão).

O otimizador necessitará de uma proximidade razoável das informações, para estimar os custos nas estratégias de execução, mas não há necessidade de elas sejam atualizadas constantemente

Relembrando...

- Número de registros (r)
- Tamanho do registro médio (R)
- Tamanho do bloco (B)
- Fator de bloco (fb = chão(B / R))
- Número de blocos do arquivo (b = teto(r / fb))
- Número de níveis (x)
- Número de blocos de índice de primeiro nível (b_i = teto(r_i / fb_i))
- Número de valores distintos de um atributo (d)
- Seletividade de um atributo (sl)
- Cardinalidade de seleção (cs = sl * r)
  - Atributo chave: d=r, sl=1/r e cs=1
  - Atributo não chave: sl=1/d e cs=r/d

Funções de Custo para a Seleção - S1 a S4

(as funções estimam os acessos a blocos)

S1 (busca linear): condição de igualdade

sobre não chave : $C_{S1a} = b$

sobre chave : $C_{S1b} = b / 2$, se encontrado registro, caso contrário b

S2 (busca binária): condição de igualdade

$C_{S2} = \log_2 b + teto(cs / fb) - 1$

S3 (busca usando índice primário para busca de único registro):

Índice primário : $C_{S3a} = x + 1$

Chave hash : $C_{S3b} = 1$

S4 (busca usando índice de ordenação para busca de vários registros):

>, >=, <, <=, em um campo chave

$C_{S4} = x + b/2$

Funções de Custo para a Seleção - S5

S5 : Índice de Clusterização para recuperar vários registros

- condição de igualdade (=)
- que cs registros satisfaçam a condição

$C_{S5} = x + \text{teto}(cs / fb)$

Funções de Custo para a Seleção - S6

S6 : Usando índice secundário (B+)

Condição de igualdade (=)

sobre chave : $C_{S6a} = x + 1$

sobre não chave : $C_{S6b} = x + cs$

Condições >, >=, <, <=, se metade dos registros satisfarão a condição, então metade dos blocos de índice de primeiro nível são acessados mais metade dos registros de arquivo via índice

$$C_{S6c} = x + (b_i / 2) + (r / 2)$$

Funções de Custo para a Seleção - S7 e S8

S7 : Seleção conjuntiva :

ou S1 ou um dos métodos de S2 até S6.

S8 : Seleção conjuntiva usando índice composto :

dependendo do tipo de índice usar:

$S3_a$ ou S5 ou $S6_a$ ou $S6_b$.

Exemplo: Funções de Custo para a Seleção

Tabela Empregado com 10.000 registros em 2.000 blocos de disco e fator de bloco fb = 5 e com os seguintes caminhos de acesso:

- Índice de clusterização sobre salário em 3 níveis de índice (x_{sal}=3) e cardinalidade de seleção cs_{sal} = 20
- Índice secundário sobre o atributo chave idE com x_{idE} = 4 e cs_{idE} = 1
- Índice secundário sobre sobre atributo não chave idDe, x_{idDe} = 2, 4 blocos de índice de 1º nível, 125 valores diferentes para idDe (d_{idDe}=125) e cs_{idDe} = (r_E / d_{idDe}) = 80
- Índice secundário sobre sexo, x_{sexo} = 1, d_{sexo} = 2, cs_{sexo} = 5000

Exemplo: Funções de Custo para a Seleção

(acessos a blocos)

a) $\sigma_{idE=1234}$ (Emp)

força bruta:

sobre chave: $C_{S1b} = b / 2$ → 2.000 / 2 = 1.000

índice secundário:

$C_{S6a} = x + 1$ → 4 + 1 = 5

b) $\sigma_{idDe>5}$ (Emp)

força bruta (não chave): $C_{S1a} = b$ → 2.000

usando $C_{S6c} = x + (b_i / 2) + (r / 2)$ → 2 + 4/2 + 10.000/2 = 5.004

c) $\sigma_{idDe=5}$ (Emp)

força bruta: $C_{S1a} = b$ → 2.000

índice secundário: $C_{S6b} = x + cs$ → 2 + 80 = 82

Exemplo: Funções de Custo para a Seleção

d) $\sigma_{\text{idDe=5 AND salario >3000 AND sexo= 'F'}}$ (Emp)

força bruta:

$C_{S1a} = b \rightarrow 2.000$

sem força bruta:

sobre idDe : S6b	→	82
sobre salario : $C_{S4} = x + b/2$	→	3 + 2.000/2 = 1.003
sobre sexo : $C_{S6b} = x + cs$	→	1 + 5.000 = 5001

Qual a ordem de execução das seleções simples?

- Começa com idDe (menor custo), depois vai para salario, por fim, sexo; onde as duas etapas finais da conjunção são realizadas em memória.

Funções de Custo para a Junção

Seletividade de junção: razão entre tamanho do arquivo de junção resultante e o tamanho do arquivo de produto cartesiano. Ou seja, é a estimativa do arquivo resultante.

$$sj = |(R >\!<_c S)| \ / \ |(R \times S)| = |(R >\!<_c S)| \ / \ |(R| \ * \ |S|)$$

Sendo $R >\!<_{R.A=S.B} S$

A é chave de R, então $|(R >\!<_c S)| \leq |S|$, de modo que $sj \leq (1 \ / \ |R|)$

- Cada registro no arquivo S será juntado com no máximo um registro no arquivo R

B é chave de S, então $|(R >\!<_c S)| \leq |R|$, de modo que $sj \leq (1 \ / \ |S|)$

Os cálculos dos custos são

- Se não há condição de junção então **sj = 1,** junção é o produto cartesiano
- Se não houver tupla de R que combine com tupla de S então **sj = 0**
- Em geral, 0 <= js <= 1

Logo, estimativa da seletividade é: $|(R >\!<_c S)| = sj * |R| * |S|$

Funções de custo para a junção

(as funções estimam os acessos a blocos)

Para a junção $R >\!<_{R.a = S.b} S$, R com b_R blocos e S com b_S blocos. Seja fb_{Res} o fator de bloco do resultado e sj a seletividade da junção.

J1(força bruta): Seja R o loop externo, com 3 buffers de memória

$C_{J1} = b_R + b_R * b_S + sj * |R| * |S| / fb_{Res}$

J2 (usando índice): R linear, S com estrutura de acesso em b com níveis de índice x_B.

a) Para índice secundário, onde S_B é a cardinalidade de seleção em *S.b*

$C_{J2a} = b_R + |R| * (x_B + s_B) + sj * |R| * |S| / fb_{Res}$

b) Para índice de clusterização, onde S_B é a cardinalidade de seleção em *S.b*

$C_{J2b} = b_R + |R| * (x_B + (s_B / fb_B)) + sj * |R| * |S| / fb_{Res}$

c) Para índice primário

$C_{J2c} = b_R + |R| * (x_B + 1) + sj * |R| * |S| / fb_{Res}$

d) chave hash *h* em S.b com número médio de acessos *h* (h >= 1)

$C_{J2d} = b_R + |R| * h + sj * |R| * |S| / fb_{Res}$

Funções de custo para a junção

J3 (ordenação-intercalação):

a) Com os arquivos já ordenados nos atributos de junção:

$C_{J3a} = b_R + b_S + sj * |R| * |S| / fb_{Res}$

b) Pelo menos um dos arquivos não ordenado, acrescer custos de ordenação do arquivo

$C_{ord} = 2 * b + 2 * b * \log_{gMerge} n_{Run}$

Exemplo: Funções de Custo para a Junção

Tabela Empregado descrita anteriormente

Tabela Departamento com 125 registros em 13 blocos com:

- Índice primário sobre idD de Departamento com 1 nível
- Índice secundário sobre idEch com dois níveis, $cs_{idEch} = 1$

Sejam as junções

- OP6 : Empregado $\bowtie_{idDe=idD}$ Departamento
- OP7 : Departamento $\bowtie_{idEch=idE}$ Empregado

Cada departamento um chefe, então $sj_{op6} = 1/125$

Admitir que o fator de bloco do resultado é $fb_{Res} = 4$

Exemplo: Funções de Custo para a Junção
Usando os métodos J1 e J2 para OP6

1-Usando J1 com Empregados como loop externo

$C_{J1} = b_E + b_E * b_D + sj_{OP6} * r_E * r_D / fb_{Res}$

$= 2000 + 2000 * 13 + 1/125 *10000 * 125 / 4 = 30.500$ blocos/acesso

2-Usando J1 com Departamento como loop externo

$C_{J1} = b_D + b_D * b_E + sj_{OP6} * r_E * r_D / fb_{Res}$

$= 13 + 13 * 2000 + 1/125 *10000 * 125 / 4 = 28.513$ blocos/acesso

3-Usando J2 com Empregados como loop externo

$C_{J2c} = b_E + r_E * (x_{idD} + 1) + sj_{OP6} * r_E * r_D / fb_{Res}$

$= 2000 + 10000 * (1+1) + 1/125 *10000 * 125 / 4 = 24.500$ blocos/acesso

4-Usando J2 com Departamento como loop externo

$C_{J2a} = b_D + r_D * (x_{idDe} + S_{idDe}) + sj_{OP6} * r_E * r_D / fb_{Res}$

$= 13 + 125 * (2+80) + 1/125 *10000 * 125 / 4 = 12.763$ blocos/acesso

A menor estimativa de custo será escolhida.

Determine custos de OP7.

Exemplo: Funções de Custo para a Junção

Se mais de 3 buffers de memória estivessem disponíveis?

Então,

J1(força bruta): Seja R o loop externo, com n buffers de memória

$C_{J1} = b_R + (teto(b_R / (n_b – 2))) * b_S + sj * |R| * |S| / fb_{Res}$

Exemplo com 15 buffers disponíveis para o caso 2

2-Usando J1 com Departamento como loop externo

$C_{J1} = b_D + (teto(b_D / (n_b – 2))) * b_E + sj_{OP6} * r_E * r_D / fb_{Res}$

$= 13 + (teto(13 / (15 - 2))) * 2000 + 1/125 *10000 * 125 / 4 = 4.513$ blocos/acesso

contra (28.513)

Otimização Consulta Semântica

Exemplo: Os nomes dos empregados quem ganham mais que seus supervisores

```
SELECT e.nomefim, s.nomefim
FROM Emp as e, Emp as s
WHERE e.idEsup = s.idE AND e.salario > s.salario
```

Se no esquema do banco de dados existe a informação de que não existe tais empregados, o analisador de consultas não perde tempo em executar essa consulta, porque já sabe que é vazia (consulta o esquema).

Reduz o tempo, mas se tivermos muitas informações desse tipo, consumirá tempo de processamento.